C++ Virtual Dispatch Under the Hood: VTable & VPtr

The VTable & VPtr Concepts

Imagine this example:

```
class Animal {
public:
    virtual void sound() {
        // animal sounds
    }
};

class Dog : public Animal {
public:
    virtual void sound() {
        // animal barks
    }
};
```

Now if we use:

```
Animal * ptr = new Dog();
ptr->sound();
// O/p -> Barks ✔
```

Vtable : Virtual function table

To understand HOW: It is a table maintained by the compiler which contains pointers to virtual functions of that object class.

- So `Animal` vtable has a pointer to `animal::sound`
- `Dog` vtable has a pointer to `dog::sound`

Vptr : Virtual table pointer

It is a hidden pointer, which is inserted into an object of a polymorphic class (i.e. at least one virtual function). Its job is to point to the appropriate vtable.

- If a parent class has a virtual function and a child inherits from the parent, then even if the child doesn't have his own virtual function, it's a polymorphic class and gets its vptr.

Step-by-Step: Virtual Dispatch

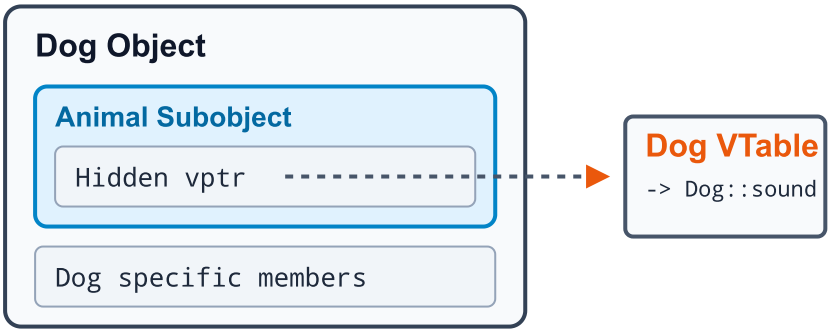
So in the code:

1. `ptr` is an `Animal*`
2. it points to the `Animal` subobject
3. the `Animal` subobject has a hidden vptr
4. THE VPTR is determined by the actual object dynamic type

So when we create `new Dog()` it inserts a vptr for Dogs vtable. This whole process is virtual dispatch.

Memory Layout: Single Inheritance

Below is the Dog object memory layout:



- Dog contains Animal subobject
- Animal subobject contains hidden vptr and Animal members
- hidden vptr points to Dog Vtable
- Dog also contains Dog specific members

Static vs Dynamic Binding

@ If we didn't have `virtual sound`, then `ptr` has static type `Animal *` and without virtual, the compiler decides which function to call using the pointer's static type.

STATIC TYPE ➔ STATIC BINDING

DYNAMIC TYPE ➔ VIRTUAL DISPATCH

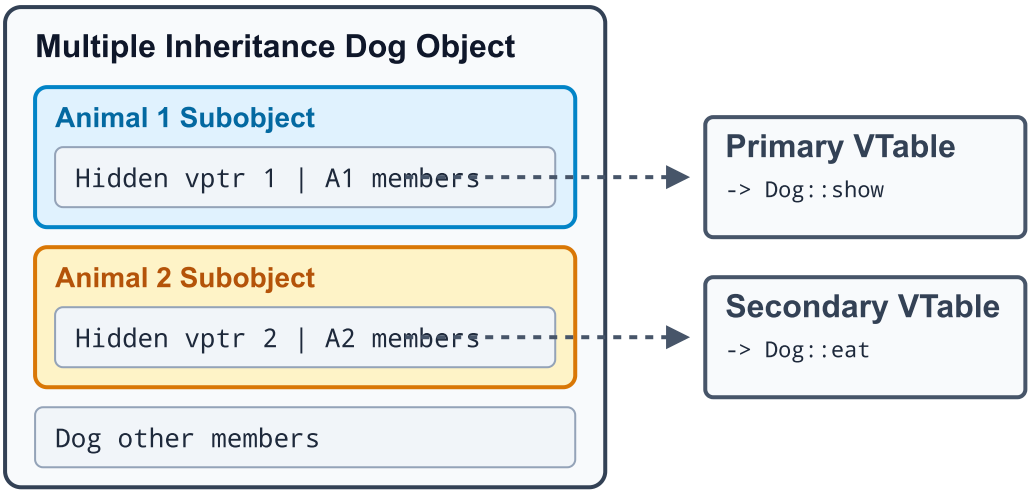
VTable & VPtr Architecture:
Vtable is not created separately for every object. 2 objects of the same class share the class's vtable. However, each object has their own vptr.

Memory Layout: Multiple Inheritance

Now imagine the case of MULTIPLE INHERITANCE:

- `animal 1` with virtual `show`
- `animal 2` with virtual `eat`
- and `dog` inherits from both and has `show` and `eat`

Now if we create a dog object, it will have 2 base class subobjects and each base class subobject will have its own vptr pointing to the relevant vtable.



- **Dog contains Animal 1**
  - Animal 1 contains hidden vptr 1 -> points to `Dog::show`
  - Animal 1 contains Animal 1 members
- **Dog contains Animal 2**
  - Animal 2 contains hidden vptr 2 -> points to `Dog::eat`
  - Animal 2 contains Animal 2 members
- **Dog contains Dog other members**